Lab Ten – Point Estimation

1 Lab 8 Notes

1.1 Altham's Multiplicative Binomial Distribution

The PMF for Altham's Multiplicative Binomial Distribution is

$$f_X(x) = \frac{1}{c}\binom{n}{x}p^x(1-p)^{n-x}\theta^{x(n-x)}I(x \in \{0,1,...,n\})$$

where

$$c = \sum_{i=0}^{n}\binom{n}{i}p^i(1-p)^{n-i}\theta^{i(n-i)}.$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```
daltbinom <- function(x, size, prob, theta){
  normalizing.constant <- 0
  for(j in 0:size){
    normalizing.constant <- normalizing.constant +
      choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
  }
  # Probability Mass at x=x
  (1/normalizing.constant) *
    choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
      (x>=0)*(x<=size)
}
```

2 Altham's Multiplicative Beta Binomial Distribution

The PMF for Altham's Multiplicative Beta Binomial Distribution is

$$f_X(x) = \left[\int_0^1 \left(\frac{1}{c}\binom{n}{x}p^x(1-p)^{n-x}\theta^{x(n-x)}\right)\left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)}p^{\alpha-1}(1-p)^{\beta-1}\right)dp\right] I(x \in \{0,1,...,n\})$$

Prof. Cipolli included `R` function for computing probability using this distribution.

```
daltbbinom.single <- function(x, size, alpha, beta, theta){
    integrand <- function(p) {
    sapply(p, # integrate will pass in a vector, so we use sapply to apply the following
           function(p_val){
             # f(x|p) * f(p|alpha, beta)
             daltbinom(x, size, p_val, theta) * dbeta(p_val, alpha, beta)
           }
    )
  }
  # Probability Mass at x=x
  # integrate(integrand, lower = 0, upper = 1)$value * (x>=0)*(x<=size)
```

```r
  # I shrink the bounds below to avoid log(0)=-infinty
  # I use try() to catch when there is an error and avoid crashing
  res <- try(integrate(integrand, lower = 1e-7, upper = 1 - 1e-7)$value *
                (x>=0)*(x<=size),
             silent = TRUE)
  # If there is an error, return a near-zero probability
  if(inherits(res, "try-error")) return(0.1^6)

  res
}
# Write a function that works on a vector
daltbbinom <- function(x, size, alpha, beta, theta){
  sapply(X=x, FUN=daltbbinom.single, size=size, alpha=alpha, beta=beta, theta=theta)
}
```

3 Question 1

In Lab 8, we use Altham's Multiplicative Binomial Distribution. Your job was to interpret the impact of θ in that equation. In this lab, we will model the number of legs won in a best of eleven (first to six) match.

To help us think about the interpretation, something that was challenging before, let's try something a little more manageable.

3.1 Part a

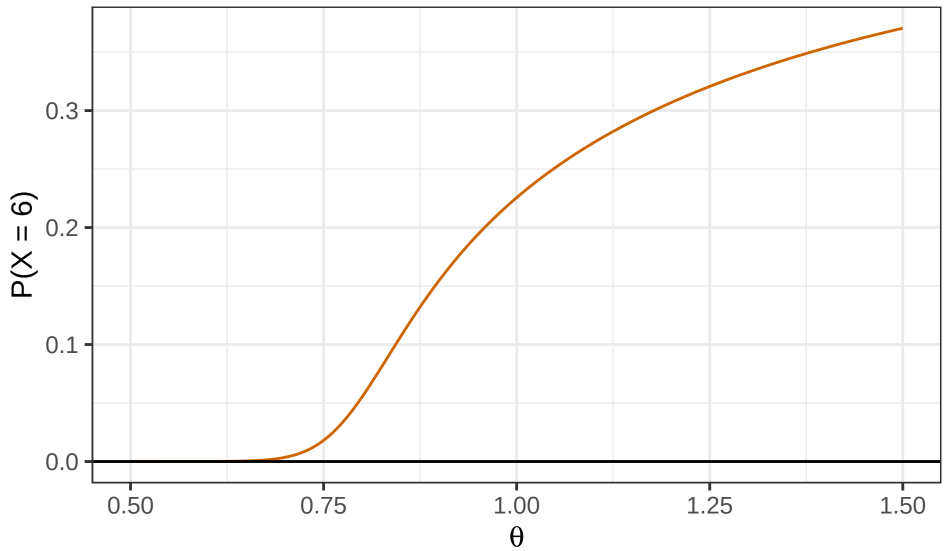
Suppose a player has a 50% chance of winning a specific leg, that is let $p = 0.50$. Let's look at the probability they win $(x = 6)$ legs in the match. Compute this probability over a sequence from $\theta = 0.5$ to $\theta = 1.5$ and plot it using a line where the x-axis is θ and the y-axis is $P(X = 6)$.

Solution

```r
theta_seq = seq(0.5, 1.5, length.out=1000)
prob_x6 = rep(NA, length(theta_seq))
for(i in 1:length(theta_seq)){
  prob_x6[i] = daltbinom(x=6, size=11, prob=0.50, theta=theta_seq[i])
}
x6 <- tibble(theta = theta_seq,
             prob  = prob_x6)
```

```r
ggplot(x6) +
  geom_line(aes(x = theta, y = prob), color = "darkorange3") +
  geom_hline(yintercept = 0) +
  theme_bw() +
  labs(x = expression(theta), y = "P(X = 6)")
```

Figure 1: The probability that the player wins the match across different θ



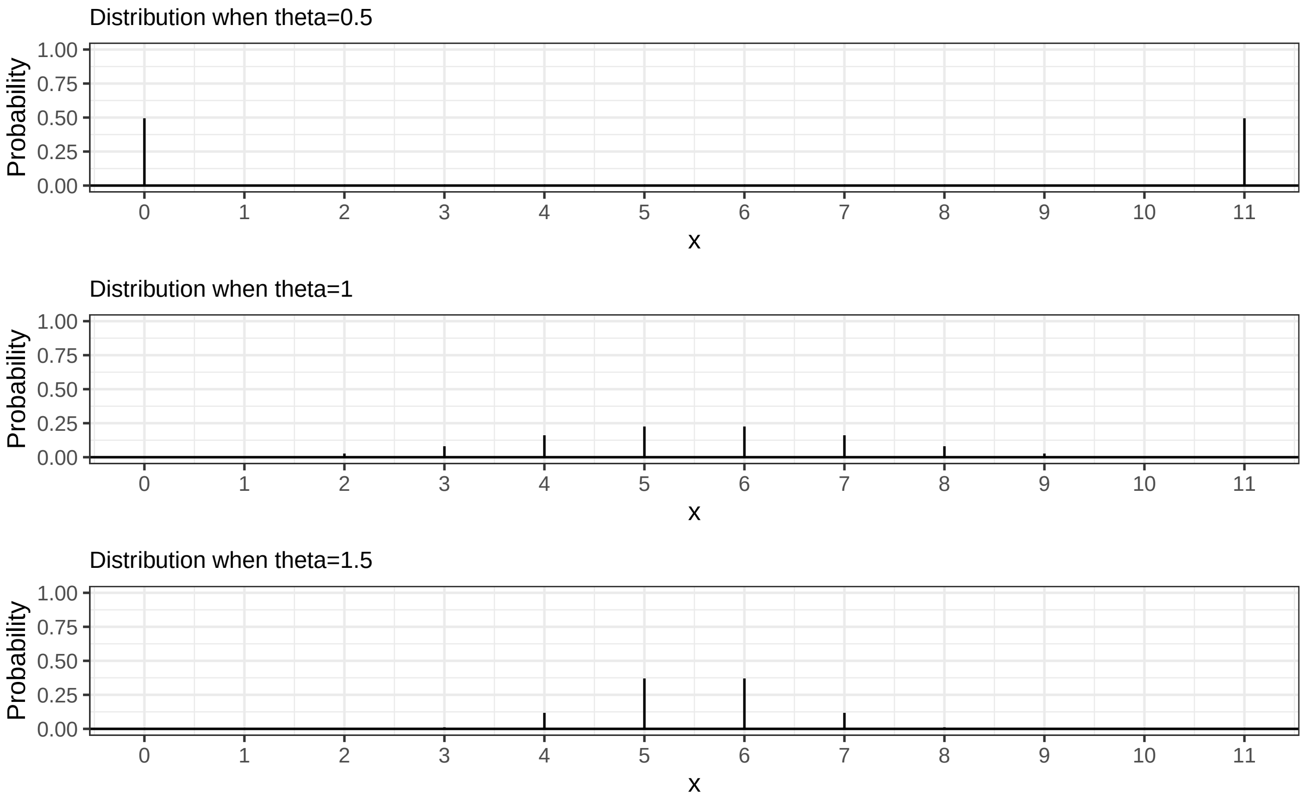
3.2 Part b

Now, plot Altham's Multiplicative Binomial Distribution under the same parameters with $\theta = 0.50$, $\theta = 1$, and $\theta = 1.5$. Use `ncol=1` in `facet_wrap()` to plot the PMFs in a single column for comparing.

Solution

```
ggdat <- tibble(x = 0:11) |>
  mutate(PMF = daltbinom(x,11,0.5,0.5))
PMF.plot.1 <- ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:11) +
  ylim(0,1)+
  ylab("Probability") +
  theme_bw() +
  ggtitle("Distribution when theta=0.5")+
  theme(plot.title = element_text(size = 10))

ggdat <- tibble(x = 0:11) |>
  mutate(PMF = daltbinom(x,11,0.5,1))
PMF.plot.2 <- ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:11) +
  ylim(0,1)+
  ylab("Probability") +
  theme_bw() +
  ggtitle("Distribution when theta=1") +
  theme(plot.title = element_text(size = 10))

ggdat <- tibble(x = 0:11) |>
  mutate(PMF = daltbinom(x,11,0.5,1.5))
PMF.plot.3 <- ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:11) +
  ylim(0,1)+
  ylab("Probability") +
  theme_bw() +
  ggtitle("Distribution when theta=1.5")+
  theme(plot.title = element_text(size = 10))

PMF.plot.1 / PMF.plot.2 / PMF.plot.3
```

Figure 2: Altham's Multiplicative Binomial Distribution across different θ



3.3 Part c

What is your best assessment of what θ is doing in this setting? Consider when a 6-0 blowout is most likely, and when a 5-6 nail-biter is most likely. Note here we should treat the match as a fixed $n = 11$ as we did for $n = 3$ in Lab 8.
Note: In lab 8, we saw that small θ meant wins came in bunches (clumped together), so there were more 2-0 wins in a best of three series.

Solution

When $\theta = 0.5$, the probability of winning 6-0 is high (wins cluster). When $\theta = 1$, it's a Binomial distribution. When $\theta = 1.5$, the player is highly likely to lose 5-6 or win 6-5 (close matches).

4 Question 2

4.1 Part a

Altham's Multiplicative Binomial Distribution does not typically simplify to the clean $E(X) = np$ seen in the Binomial distribution, unless $\theta = 1$. Write out the expression that would need to be solved to find $E(X^k)$, and then write a function that computes it given the proposed parameter values p (`prob`) and θ (`theta`).

Solution

$$E(X^k) = \sum_{x=0}^{n} x^k \cdot f_X(x)$$

```
kth_moment <- function(k, size, prob, theta){
  normalizing.constant <- 0
  for(j in 0:size){
    normalizing.constant <- normalizing.constant +
```

```
    choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
  }
  # E(X^k)
  result <- 0
  for (x in 0:size){
    result <- result +
    x^k * (1/normalizing.constant) *
    choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
      (x>=0)*(x<=size)
  }
  print(result)
  }
```

4.2 Part b

Describe how you can use your answer to part a to find Method of Moments estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution MOM sets sample moments equal to population moments. Since the Altham's Multiplicative Binomial Distribution has 2 unknown parameters, we'll need to solve these 2 equations to find p and θ:

$$\bar{x} = E(X^1) = x^k \cdot f_X(x)$$

$$E(X^2) = x^2 \cdot f_X(x)$$

4.3 Part c

Write a function that computes the log-likelihood for Altham's Multiplicative Binomial Distribution, given data (`x`) and proposed parameter values p (`prob`) and θ (`theta`).

Solution

```
lldaltbinom <- function(prob, theta, x, size, mle = F) {
  # ensuring p is between 0 and 1
  prob <- ifelse(mle, 1/(1+exp(-prob)), prob)
  # ensuring theta is positive
  theta <- ifelse(mle, exp(theta), theta)

  ll <- sum(log(daltbinom(x, size, prob, theta)))
  # if mle is true (we're trying to find mle), return the negative log-likelihood.
  # if it's false, return the log-likelihood)
  ifelse(mle, -ll, ll)
}
```

4.4 Part d

Describe how you can use your answer to part c to find Maximum Likelihood estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution Find the values of p and θ that maximize the log-likelihood by using `optim`.

5 Lab 10

Peter "Snakebite" Wright is a two-time Professional Darts Corporation World Champion (2020, 2022) and a former World No. 1. He is one of the sport's most decorated players, winning nearly 50 PDC titles including the World Matchplay, UK Open, and multiple European Championships.

In 2024, he made The Premier League – a league involving the eight best players that runs every Thursday night from February to May. In this season, he won two of eighteen matches, securing only four points. The spread in legs won was -43, meaning he lost 43 more legs than he won. This performance was rather quite bad. He was soundly trounced in almost all his match ups.

The 2024 standings are below:

Rank	Player
1	Luke Littler
2	Luke Humphries
3	Michael van Gerwen
4	Michael Smith
5	Nathan Aspinall
6	Rob Cross
7	Gerwyn Price
8	Peter Wright

5.1 Summarize the 2024 Data

In `pdc2024.csv`, I have provided the data for the 2024 Premier League season. There are a few data cleaning steps to consider.

- Remove any matches with no `Score` (e.g., a bye or walkover)
- Nights 1 through 16 are best of 11 (first to 6), but the playoffs are extended. For consistency, keep nights 1 through 16 only.
- Make two columns `WinnerLegs` and `LoserLegs` that keep track of the number of legs each player has won

Summarize the data. What do the data tell us about the breakdown of the results?

Solution

```r
pdc = read_csv("pdc2024.csv")
pdc.cleaned <- pdc|>
  filter(Score!= "w/o",
        Night != "Finals")|>
  separate(Score, into = c("WinnerLegs", "LoserLegs"))
```

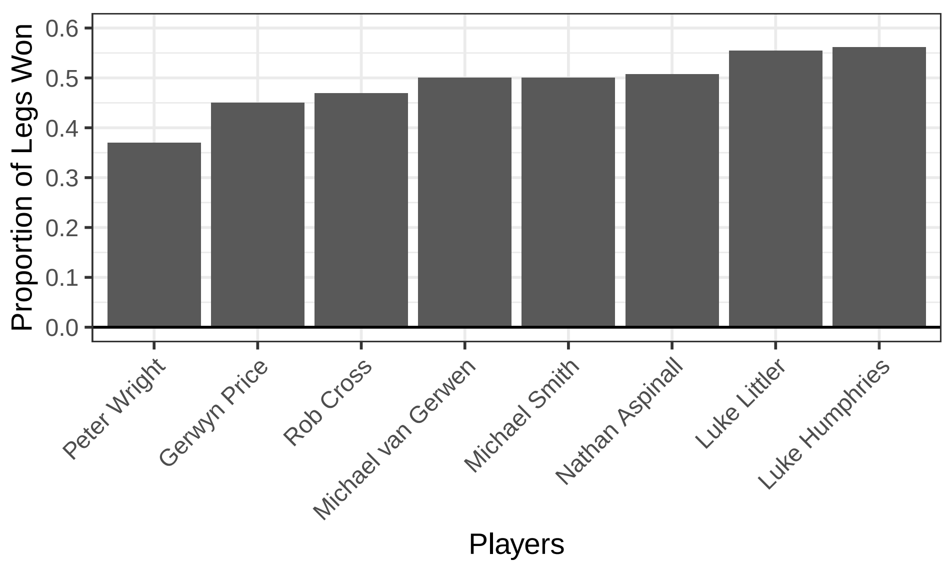
```r
playername = c("Luke Littler", "Luke Humphries", "Michael van Gerwen", "Michael Smith",
              "Nathan Aspinall", "Rob Cross", "Gerwyn Price", "Peter Wright")
legswon = rep(NA, 8)
legslost = rep(NA, 8)

for (i in 1:length(playername)){
  # even when they lose, they still win some legs
  win = sum(as.numeric(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner==playername[i]])) +
        sum(as.numeric(pdc.cleaned$LoserLegs[pdc.cleaned$Loser==playername[i]]))
  lost = sum(as.numeric(pdc.cleaned$WinnerLegs[pdc.cleaned$Loser==playername[i]])) +
         sum(as.numeric(pdc.cleaned$LoserLegs[pdc.cleaned$Winner==playername[i]]))
  legswon[i] <- win
  legslost[i] <- lost
}

player.stats = tibble(
  Player = playername,
  LegsWon = legswon,
  LegsLost = legslost)

player.stats <- player.stats |>
  mutate(TotalLegs = LegsLost + LegsWon) |>
  mutate(PropWin = LegsWon / TotalLegs) |>
  mutate(LegsSpread = LegsWon - LegsLost)
```

Table 1: Number of Legs Won and Lost for Each Player

Player	LegsWon	LegsLost	TotalLegs	PropWin	LegsSpread
Luke Littler	193	155	348	0.55	38
Luke Humphries	169	132	301	0.56	37
Michael van Gerwen	135	135	270	0.50	0
Michael Smith	150	150	300	0.50	0
Nathan Aspinall	143	139	282	0.51	4
Rob Cross	115	130	245	0.47	-15
Gerwyn Price	95	116	211	0.45	-21
Peter Wright	61	104	165	0.37	-43

```r
ggplot(player.stats |>
          mutate(Player = factor(Player,
                                  levels= player.stats$Player[order(player.stats$PropWin)]))) +
  geom_bar(aes(x=Player,
               y=PropWin),
          stat = "identity") +
  geom_hline(yintercept = 0) +
  theme_bw() +
  labs(x = "Players",
        y = "Proportion of Legs Won") +
  scale_y_continuous(breaks = seq(0, 0.6, 0.1), limits = c(0, 0.6)) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Figure 3: Proportion of Legs Won for Each Player




```r
pdc.score <- pdc |>
  filter(Score!= "w/o",
         Night != "Finals")|>
  group_by(Score) |>
  summarize(n = n()) |>
  mutate(Proportion = n/sum(n))
```

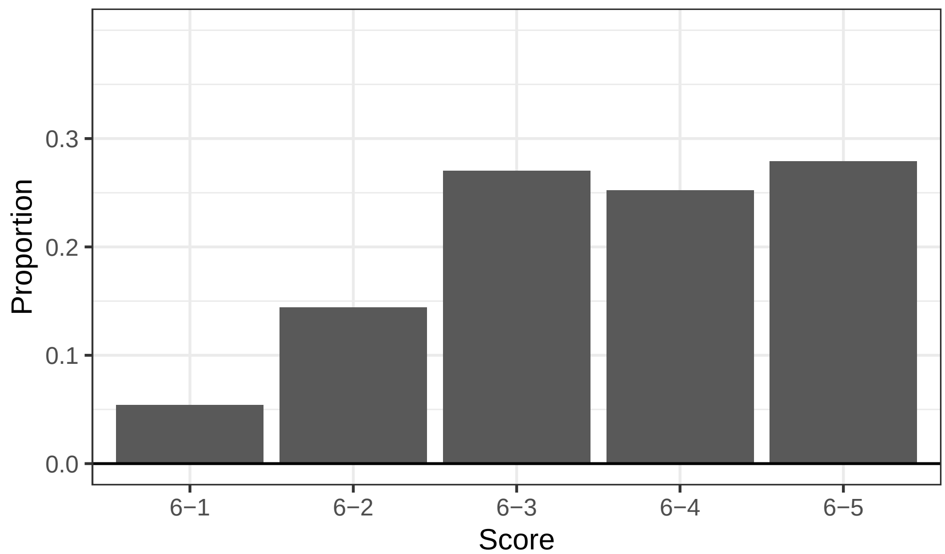
Table 2: Score for Each Match

Score	n	Proportion
6-1	6	0.05
6-2	16	0.14
6-3	30	0.27
6-4	28	0.25
6-5	31	0.28

```r
ggplot(pdc.score) +
  geom_col(aes (x= Score,
```

```
                 y = Proportion)) +
  geom_hline(yintercept = 0) +
  scale_y_continuous(breaks = seq(0, 0.3, 0.1), limits = c(0, 0.4)) +
  theme_bw()+
  labs (x = "Score",
        y = "Proportion")
```

Figure 4: Proportion of Score



Luke Humphries has the highest proportion of legs won, followed closely by Luke Littler and Nathan Aspinall. From the table, we can see that Gerwyn Price, Rob Cross, and Peter Wright had more legs lost than legs won (meaning their leg spread are negative). Michael van Gerwen and Michael Smith have leg spreads of 0.

Most of the matches are won by 6-5 and 6-3, meaning that most people win by close margin instead of sweeping.

5.2 Altham's Multiplicative Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maximium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLE for the player using this data. Ensure to report p and θ for each player in a nicely formatted table.
  Note: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space. For example, here, you might consider `p <- 1/(1+exp(-par[1]))` and `theta <- exp(par[2])`.
- Make comments about the MLEs in the context of the standings.

Solutions

Create vectors

```
# use list because vectors are of different lengths
wins = list(
  LT = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Luke Littler"],
                  pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Luke Littler"])),
  LH = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Luke Humphries"],
                  pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Luke Humphries"])),
  MG = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Michael van Gerwen"],
                   pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Michael van Gerwen"])),
```

```
  MS = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Michael Smith"],
                    pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Michael Smith"])),
  NAs = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Nathan Aspinall"],
                    pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Nathan Aspinall"])),
  RC = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Rob Cross"],
                    pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Rob Cross"])),
  GP = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Gerwyn Price"],
                    pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Gerwyn Price"])),
  PW = as.numeric(c(pdc.cleaned$WinnerLegs[pdc.cleaned$Winner=="Peter Wright"],
                    pdc.cleaned$LoserLegs[pdc.cleaned$Loser=="Peter Wright"]))
)
```

Finding MLEs

```
# leave functions out of the loop
daltbinom <- function(x, size, prob, theta){
  normalizing.constant <- 0
  for(j in 0:size){
    normalizing.constant <- normalizing.constant +
      choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
  }
  (1/normalizing.constant) *
    choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
      (x>=0)*(x<=size)
}

lldaltbinom <- function(par, data, mle = F){
  prob  <- ifelse(mle, 1/(1+exp(-par[1])), par[1])
  theta <- ifelse(mle, exp(par[2]), par[2])

  ll <- sum(log(daltbinom(data, size=11, prob = prob, theta = theta)))
  ifelse(mle, -ll, ll)
}
# create tibble to store data
results <- tibble(
  Player = c("Luke Littler", "Luke Humphries", "Michael van Gerwen", "Michael Smith",
             "Nathan Aspinall", "Rob Cross", "Gerwyn Price", "Peter Wright"),
  Probability = rep(NA, 8),
  Theta = rep(NA, 8))

# loop
for(i in 1:length(wins)){
  mle <- optim(par = c(0,0),
             fn = lldaltbinom,
             data= wins[[i]],
             mle=T)
results$Probability[i] <- 1/(1+exp(-mle$par[1]))
results$Theta[i]<- exp(mle$par[2])
}
```

Table 3: p and θ for each player

Player	Probability	Theta
Luke Littler	0.47	1.32
Luke Humphries	0.46	1.20
Michael van Gerwen	0.43	1.02
Michael Smith	0.43	1.05
Nathan Aspinall	0.40	1.19
Rob Cross	0.40	1.01
Gerwyn Price	0.38	1.02

Table 3: p and θ for each player

Player	Probability	Theta
Peter Wright	0.32	0.99

When we treat each leg as independent from each other $\theta = 1$, the probability of winning each leg for each player is all less than 0.5, with Luke Littler and Luke Humphries having the highest probability of winning each leg. They indeed ended up having high proportions of legs won and high standings overall. Their θ are all larger than 1, meaning legs won are less likely to clump together than spread out (wins of 6-5 or 6-3 are more likely), which is consistent with data summary. Peter Wright has θ of almost 1, meaning that the distribution of his probability of winning each leg is a binomial distribution, which means his legs are almost independent of each other.

5.3 Altham's Multiplicative Beta Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maxmium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLEs for the player using this data. Ensure to report α, β, and θ for each player in a nicely formatted table.
  Note 1: Due to the computational complexity of `integrate()`, you should compute the logged probability for `0:6` once and add them according to the data in each player's vector. **Note 2**: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space.
- Plot the underlying Beta(α, β) distribution for each player.
  Note: Add the mean and variance of the Beta to the table of MLEs. Recall

$$E(X) = \frac{\alpha}{\alpha+\beta}$$

$$sd(X) = \sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}$$

- Make comments about the MLEs in the context of the standings.
  Note: The players alternate who throws first in a leg has the mathematical "head start" to reach a finish before their opponent can take another turn. Players alternate who throws first with each leg.

They start with 500 points, and then points are subtracted. So if a player plays well, it's almost impossible to lose.

Solution

```r
daltbbinom <- function(x, size, alpha, beta, theta){
  sapply(X=x, FUN=daltbbinom.single, size=size, alpha=alpha, beta=beta, theta=theta)
}

lldaltbbinom <- function(par, data, mle = F){
  alpha  <- ifelse(mle, exp(par[1]), par[1])
  beta <- ifelse(mle, exp(par[2]), par[2])
  theta <- ifelse(mle, exp(par[3]), par[3])
  # calculate 7 times for each player instead of n times
  p <- daltbbinom(0:6, size=11, alpha, beta, theta)
  # the first location. data (x) is 0-6
  ll <- sum(log(p[data+1]))
  ifelse(mle, -ll, ll)
}
# create tibble to store data
results.2 <- tibble(
  Player = c("Luke Littler", "Luke Humphries", "Michael van Gerwen", "Michael Smith",
             "Nathan Aspinall", "Rob Cross", "Gerwyn Price", "Peter Wright"),
```

```
  Alpha = rep(NA, 8),
  Beta = rep(NA, 8),
  Theta = rep(NA, 8),
  # name of column can't have space
  Mean = rep(NA, 8),
  SD = rep(NA, 8))
```

```
# loop
for(i in 1:length(wins)){
  mle <- optim(par = c(0,0,0),
            fn = lldaltbbinom,
            # use [[]] instead of [] because it's a list
            data= wins[[i]],
            mle=T)
  # extract before using
  a <- exp(mle$par[1])
  b  <- exp(mle$par[2])
  # calculate parameters of distributions
  mean = a/(a+b)
  sd = sqrt((a*b)/((a+b)^2*(a+b+1)))
results.2$Alpha[i] <- exp(mle$par[1])
results.2$Beta[i] <- exp(mle$par[2])
results.2$Theta[i]<- exp(mle$par[3])
results.2$Mean[i] <- mean
results.2$SD[i] <- sd
}
```

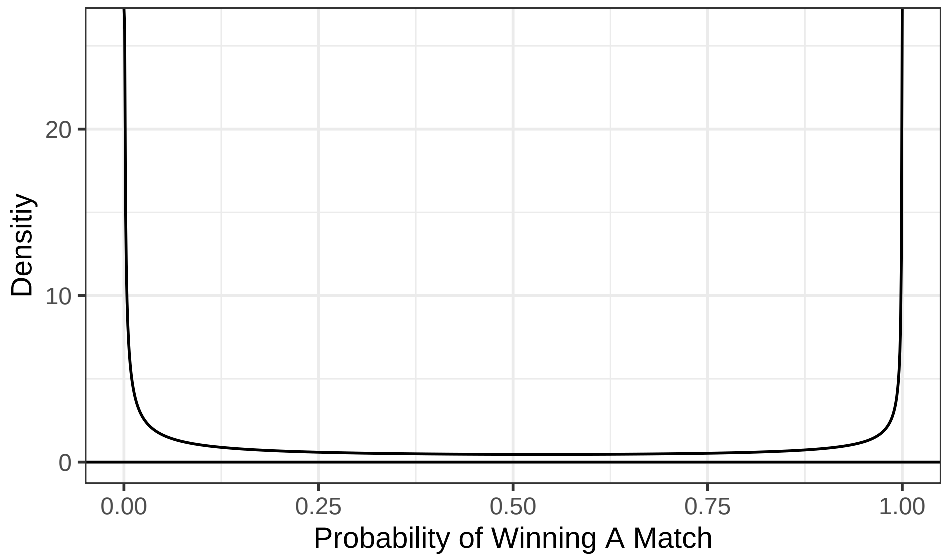
Table 4: α, β, and θ for each player

Player	Alpha	Beta	Theta	Mean	SD
Luke Littler	0.28	0.38	10.80	0.42	0.38
Luke Humphries	0.33	0.49	6.39	0.40	0.36
Michael van Gerwen	0.22	0.45	5.00	0.33	0.36
Michael Smith	0.53	1.07	2.24	0.33	0.29
Nathan Aspinall	2.47	4.34	1.49	0.36	0.17
Rob Cross	0.27	0.81	3.20	0.25	0.30
Gerwyn Price	0.26	0.92	3.18	0.22	0.28
Peter Wright	1.79	7.29	1.21	0.20	0.13

```
plots <- list()

for(i in 1:length(wins)){
  # probability ranges from 0 to 1
  ggdat <- tibble(x = seq(0,1,length.out=1000)) |>
    # R has dbeta
    mutate(PDF = dbeta(x, results.2$Alpha[i], results.2$Beta[i], ncp=0, log=F))

plots[[i]] <- ggplot() +
  geom_line(data = ggdat,
               aes(x = x, y = PDF)) +
  geom_hline(yintercept = 0) +
  xlab("Probability of Winning A Match") +
  ylab("Densitiy") +
  theme_bw()
}
```

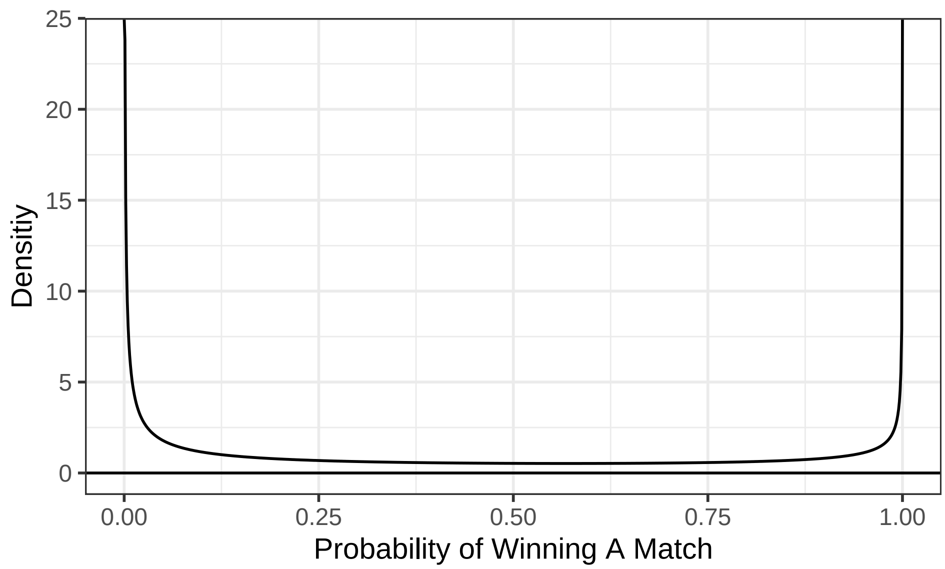
```
plots[[1]]
```

Figure 5: Probability of winning a match for Luke Littler



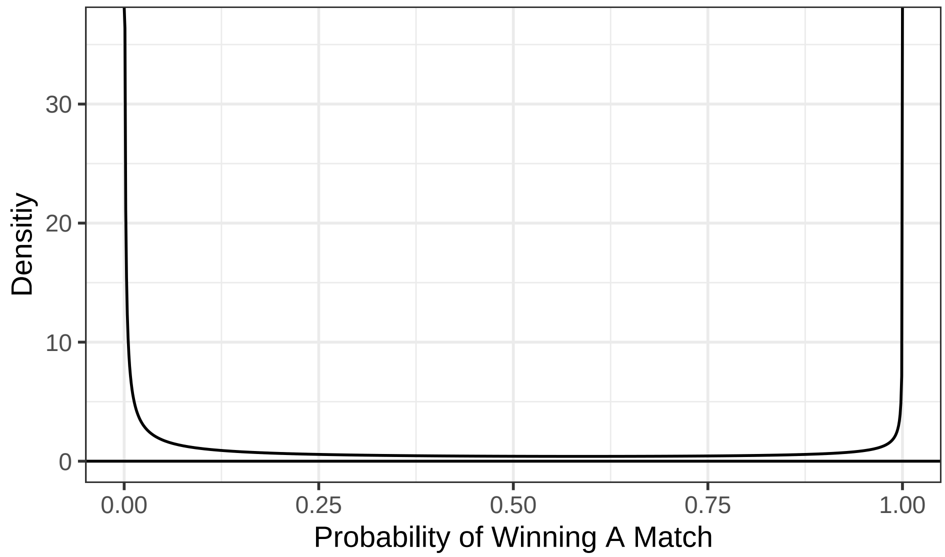

```
plots[[2]]
```

Figure 6: Probability of winning a match for Luke Humphries



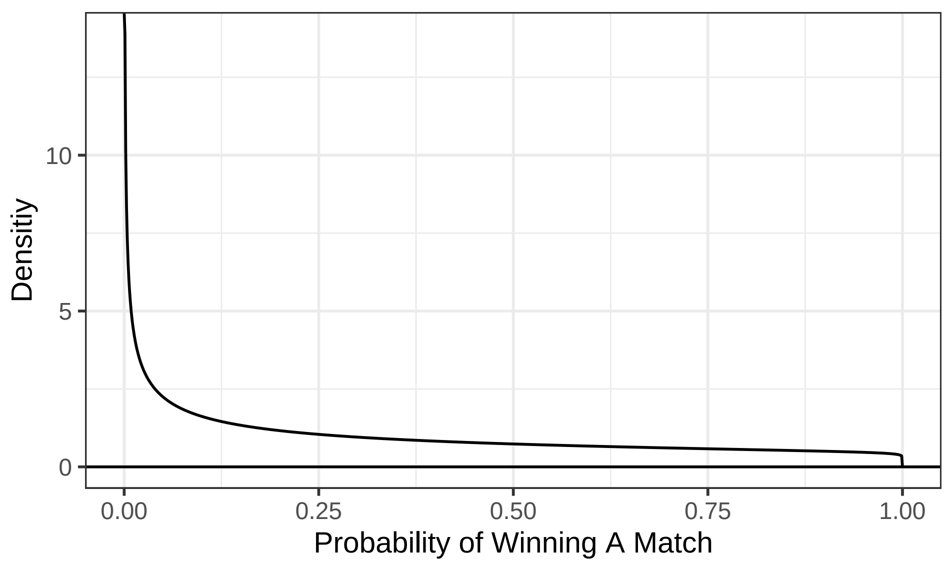

```
plots[[3]]
```

Figure 7: Probability of winning a match for Michael van Gerwen



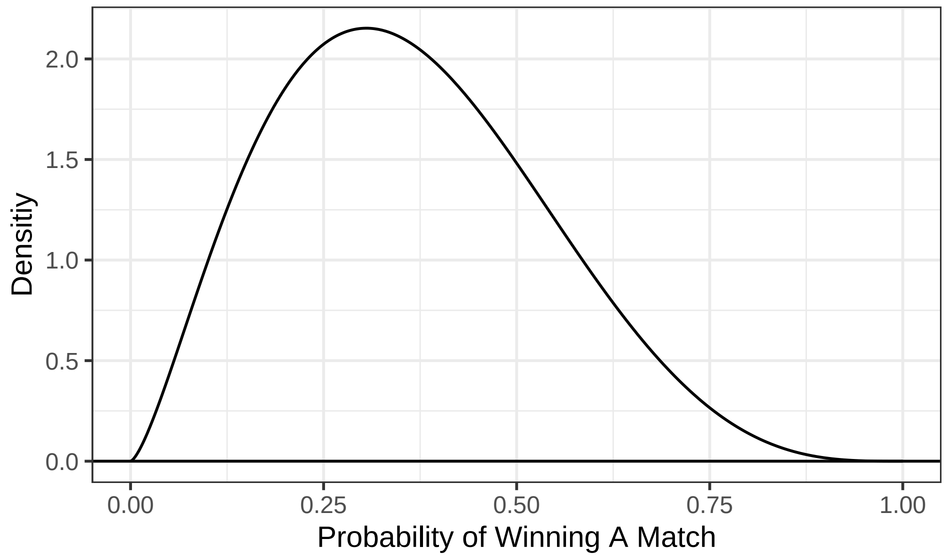

```
plots[[4]]
```

Figure 8: Probability of winning a match for Michael Smith



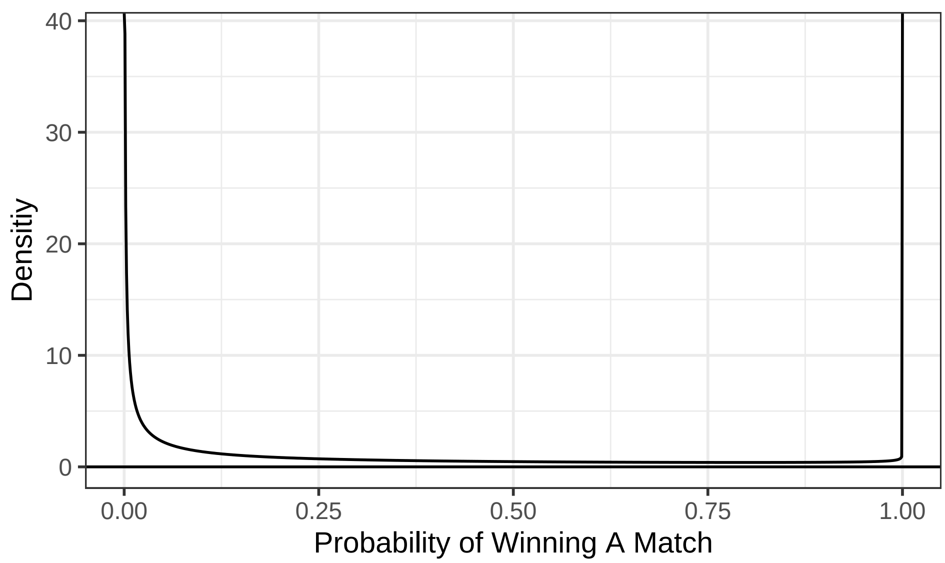

```
plots[[5]]
```

Figure 9: Probability of winning a match for Nathan Aspinall



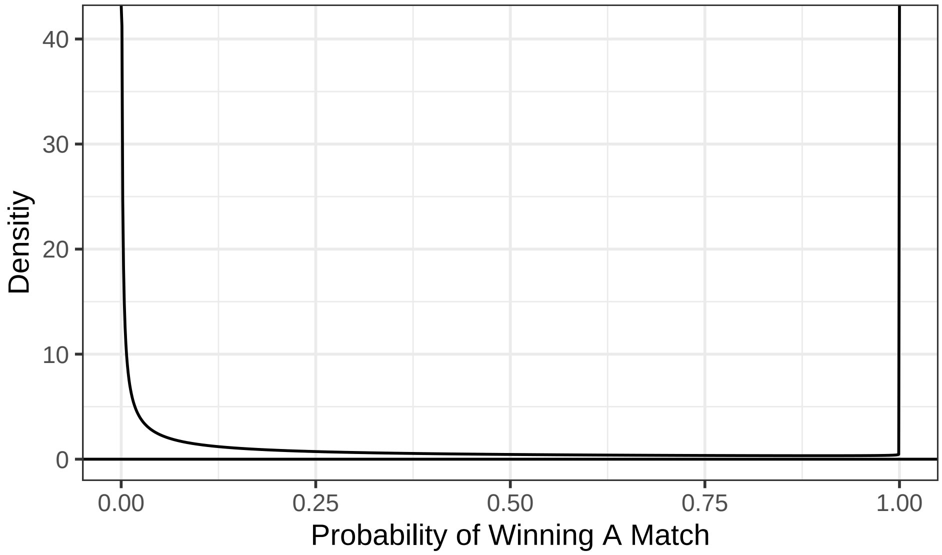

```
plots[[6]]
```

Figure 10: Probability of winning a match for Rob Cross



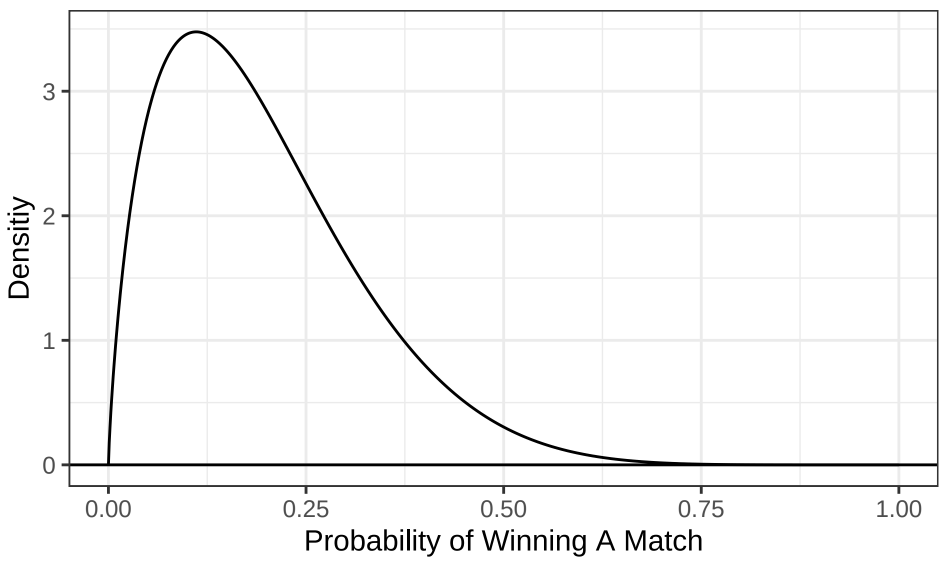

```
plots[[7]]
```

Figure 11: Probability of winning a match for Gerwyn Price




```
plots[[8]]
```

Figure 12: Probability of winning a match for Peter Wright



The beta distribution is a distribution of the probability of winning each leg for each player. Player 8, Peter Wright, have a right-skewed and more narrow beta distribution. This means that his probability of winning each leg is much less variable and consistently low. The probability of Peter Wright having a 50% chance of winning 1 leg is very low. The probability of winning each leg for Player 5, Nathan Aspinall, has a slightly right-skewed Beta distribution.

For the rest of the players, the Beta distribution plots for has a U-shape (high density near 0 and 1). This means that the probability of winning each leg is highly variable, either really high or really low. Notably, Luke Littler has $\theta = 10.80$ and Luke Humphries has $\theta = 6.39$, meaning their legs won alternate and almost don't cluster–they often play very competitive, close matches.

5.4 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) data-driven summary of the 2024 Premier League season using your work in Lab 10. Your summary should be professional, clear, and all claims should be corroborated with statistical support.

Solution

The 2024 Premier League featured 8 best dart players, who competed in best-of-11-leg matches. Each player starts out with 501 points, and as they throw, points are subtracted, so the player who throws first in a leg has the mathematical "head start" to reach a finish before their opponent takes another turn.

Luke Littler and Luke Humphries are the most dominant players. Excluding playoffs, they won 55% and 56% of total legs played respectively. The estimates of their probability of winning each leg, as modelled by Altham's Multiplicative Binomial Distributions, are 0.46 and 0.47, respectively (adjusted for uncertainty in players' skills). Peter Wright was the worst player of the season, winning only 37% of legs played and finishing with a leg spread of -43. Based on his Altham's Multiplicative Beta Binomial Distribution, his probability of winning each leg is consistently low.

Over half (53%) of the matches have a score of 6-5 or 6-4, meaning most of them are close and competitive and players only win in the second-to-last or last leg. This is expected since the league is composed of world-class players.

Looking at the Altham's Multiplicative Beta Binomial Distributions, except for Peter Wright and Nathan Aspinall, the probability of winning each leg for each player is highly variable. Thus, some nights they are very likely to win the match, while some nights it's very competitive.